XRPL Validator — Daily Engineering Report · 2026-09-07

Period: 2026-09-07 00:00 to 24:00 US Eastern (2026-09-07 04:00 to 2026-09-08 04:00 UTC); ledgers #106,815,062 to #106,837,376. Every count from the ledger and the validator's logs stops at #106,837,376. Generated 2026-10-21 00:47 EDT by report generator v3.0. Sources: the XRP Ledger (ledger headers and amendments, read from a reference rippled node), the validator's signing log and receipt logs, the deployment pipeline's gate, soak and health records, the git history of xrpl-validator and the XRPL Rust SDK, and data.xrpl.org. Scope: XRPL work only.

Revised on 2026-10-21 in report format v3; the original is kept in this repository's history. Built only from records that exist for this day. Available: the day's first and last ledger, from a public full-history server; Step 1 receipt records (they begin 2026-08-29); xrpl-validator merge records (they begin 2026-07-04); amendment data (carried from the original report); data.xrpl.org agreement scores. Not available: ledger headers (the reference node holds ledgers from 2026-10-04 22:52 UTC); signing records (they begin 2026-09-19); Steps 2–5 receipt records (they begin 2026-10-09); deployment records (they begin 2026-09-11).

Status

Not assessed — no signing records for this day.

Criteria. Normal: coverage outside planned maintenance at or above the 99.9% objective, every validation agreed with the network's validated ledger, and no engine receipt is open. Degraded: coverage below the objective, or an open engine receipt. Impaired: any validation that disagreed with the network, or coverage below 90%.

Executive summary

- **Validator:** no signing records for this day (they begin 2026-09-19).
- **Native engine:** 59 engine receipts recorded (30 fixed, 0 explained, 29 without a recorded fix).
- **Changes:** 0 deployments, 36 commits merged into main, 28 engine findings fixed (F198–F226).

Service levels

No signing records for this day (they begin 2026-09-19).

External agreement, data.xrpl.org (UTC day 2026-09-07): 0.85070 (139 missed of 931). *data.xrpl.org scores UTC days from the validations its crawlers received.*

Native engine conformance

Ledgers with a receipt in the period (#106,815,062–#106,837,376), by step:

Step	Ledgers with a receipt
Step 1 · native engine	59
Step 2 · records	0
Step 3 · order	0
Step 4 · peer data	0
Step 4 · validations	0
Step 5 · own hash	0
Writer	0

Live counters are kept only for the current run; none exist for a past day.

Receipt register

ID	Time	Ledger(s)	Step	Description	Disposition
R-01	—	#106,815,089	Step 1 · native engine	1 transaction result(s) differed from rippled's; 1 change(s) of ours the network did not make	no disposition recorded
R-02	—	#106,815,131	Step 1 · native engine	1 transaction result(s) differed from rippled's; 1 change(s) of ours the network did not make	no disposition recorded
R-03	—	#106,815,274	Step 1 · native engine	11 object(s) differed in content	fixed: F203, commit 955de48, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-04	—	#106,815,284	Step 1 · native engine	1 transaction result(s) differed from rippled's; 1 change(s) of ours the network did not make	no disposition recorded
R-05	—	#106,815,773	Step 1 · native engine	1 object(s) differed in content	fixed: F206, commit 7aaf827, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-06	—	#106,818,089	Step 1 · native engine	1 object(s) differed in content	fixed: F206, commit 7aaf827, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-07	—	#106,818,113	Step 1 · native engine	1 object(s) differed in content	fixed: F206, commit 7aaf827, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-08	—	#106,818,224	Step 1 · native engine	1 object(s) differed in content	fixed: F206, commit 7aaf827, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-09	—	#106,818,228	Step 1 · native engine	1 object(s) differed in content	fixed: F206, commit 7aaf827, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-10	—	#106,818,233	Step 1 · native engine	1 object(s) differed in content	fixed: F206, commit 7aaf827, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-11	—	#106,818,234	Step 1 · native engine	1 object(s) differed in content	fixed: F206, commit 7aaf827, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-12	—	#106,818,252	Step 1 · native engine	1 object(s) differed in content	no disposition recorded
R-13	—	#106,818,300	Step 1 · native	1 object(s) differed in content	fixed: F206, commit 7aaf827, deployed in cycle 121 (2026-09-11

ID	Time	Ledger(s)	Step	Description	Disposition
			engine		17:39 UTC)
R-14	—	#106,819,769	Step 1 · native engine	3 change(s) of ours the network did not make; 5 object(s) differed in content	no disposition recorded
R-15	—	#106,822,626	Step 1 · native engine	1 object(s) differed in content	fixed: F208, commit bdc467e, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-16	—	#106,823,197	Step 1 · native engine	2 object(s) differed in content	fixed: F209, commit f0642ec, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-17	—	#106,823,210	Step 1 · native engine	2 object(s) differed in content	fixed: F210, commit a2b29db, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-18	—	#106,823,225	Step 1 · native engine	1 object(s) differed in content	fixed: F210, commit a2b29db, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-19	—	#106,823,313	Step 1 · native engine	3 object(s) differed in content	no disposition recorded
R-20	—	#106,823,726	Step 1 · native engine	3 object(s) differed in content	no disposition recorded
R-21	—	#106,823,771	Step 1 · native engine	2 object(s) differed in content	fixed: F211, commit b5d35c7, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-22	—	#106,823,772	Step 1 · native engine	1 transaction result(s) differed from rippled's; 1 ledger change(s) the network made were absent from ours: 9143A32D	fixed: F211, commit b5d35c7, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-23	—	#106,823,807	Step 1 · native engine	2 object(s) differed in content	fixed: F212, commit 72af27a, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-24	—	#106,823,809	Step 1 · native engine	1 object(s) differed in content	fixed: F213, commit a6bd0eb, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-25	—	#106,823,826	Step 1 · native engine	3 object(s) differed in content	no disposition recorded
R-26	—	#106,823,836	Step 1 · native engine	1 object(s) differed in content	no disposition recorded
R-27	—	#106,823,837	Step 1 · native engine	1 transaction result(s) differed from rippled's; 2 ledger change(s) the network made were absent from ours: 9143A32D, 05DC8EC9	no disposition recorded
R-28	—	#106,823,869	Step 1 · native engine	1 object(s) differed in content	no disposition recorded
R-29	—	#106,823,870	Step 1 · native engine	1 transaction result(s) differed from rippled's; 2 ledger change(s) the network made were absent from ours: 05DC8EC9, 9143A32D	no disposition recorded
R-30	—	#106,823,884	Step 1 · native engine	6 object(s) differed in content	fixed: F215, commit 70d0675, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-31	—	#106,823,887	Step 1 · native engine	4 change(s) of ours the network did not make; 2 object(s) differed in content	fixed: F215, commit 70d0675, deployed in cycle 121 (2026-09-11 17:39 UTC)

ID	Time	Ledger(s)	Step	Description	Disposition
R-32	—	#106,823,899	Step 1 · native engine	1 object(s) differed in content	fixed: F214, commit fd432ed, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-33	—	#106,823,900	Step 1 · native engine	1 transaction result(s) differed from rippled's; 1 ledger change(s) the network made were absent from ours: 9143A32D	no disposition recorded
R-34	—	#106,823,926	Step 1 · native engine	1 object(s) differed in content	no disposition recorded
R-35	—	#106,823,927	Step 1 · native engine	1 transaction result(s) differed from rippled's; 1 ledger change(s) the network made were absent from ours: 9143A32D	no disposition recorded
R-36	—	#106,823,941	Step 1 · native engine	1 object(s) differed in content	no disposition recorded
R-37	—	#106,823,945	Step 1 · native engine	1 object(s) differed in content	no disposition recorded
R-38	—	#106,824,781	Step 1 · native engine	3 object(s) differed in content	fixed: F217, commit 1e805e5, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-39	—	#106,824,803	Step 1 · native engine	2 object(s) differed in content	fixed: F218, commit 7163cdc, deployed in cycle 121 (2026-09-11 17:39 UTC)
R-40	—	#106,824,971	Step 1 · native engine	3 object(s) differed in content	no disposition recorded

A receipt is recorded whenever the validator's own work differs from the network's, or the network's trusted validations reached it short of quorum. One ledger flagged by several steps is one receipt. Dispositions: fixed (with the finding, commit and deployment), explained (not a defect, with the reason), or open. Receipt and incident IDs are numbered per report: R-01 here is R-2026-09-07-01, INC-01 is INC-2026-09-07-01. Times are not shown where the reference node no longer holds the ledger.

19 more receipts (#106,825,264–#106,835,966) are in the record and not listed individually; their dispositions: 8 fixed, 11 no disposition recorded.

Incidents

Not assessed: no signing records for this day.

Change log

Deployments

No deployment records before 2026-09-11.

Merged into main

Merged	Repo	Commit	Subject
04:30 UTC	xrpl-validator	`c1b1e51`	fix(xrpl-ledger): F198 — the direct walk's tail pool turn iterates like rippled's flow()
05:16 UTC	xrpl-validator	`79395eb`	fix(xrpl-ledger): F199 — TrustSet refuses tfSetNoRipple on a negative balance
05:57 UTC	xrpl-validator	`a8439a9`	fix(xrpl-ledger): F200 — an in-limited pool slice delivers no more than the remaining want
07:49 UTC	xrpl-validator	`7c49aa8`	fix(xrpl-ledger): F201 — consecutive self-offers are swept in the pass that is still walking
08:34 UTC	xrpl-validator	`5182de9`	fix(xrpl-ledger): F202 — the bridged walk's remaining want is the entry want less the ascending fold of its saved outs
08:43 UTC	xrpl-validator	`0c4d063`	test(xrpl-node): close the F201 vector test — the F201/F202 merge resolution dropped its brace
09:37 UTC	xrpl-validator	`955de48`	fix(xrpl-ledger): F203 — a payment strand's quality function takes a pool only where its fee-inclusive spot beats the tip
10:20 UTC	xrpl-validator	`888ecec`	fix(xrpl-ledger): F204 — an input-limited payment strand drives its middle hops by the carry
11:03 UTC	xrpl-validator	`2b27ffd`	fix(xrpl-ledger): F205 — a bridge admitted by its multipath bound runs only if its single-path bound clears the limit too
11:45 UTC	xrpl-validator	`7aaf827`	fix(xrpl-ledger): F206 — a pool slice's quality is getRate-rounded, as rippled's Quality{amounts}
12:49 UTC	xrpl-validator	`bdc467e`	fix(xrpl-ledger): F208 — a crossing's pool turn is bounded by the taker's live line
13:35 UTC	xrpl-validator	`f0642ec`	fix(xrpl-ledger): F209 — an IOU difference at Number's cusp rounds the coarse mantissa
13:35 UTC	xrpl-validator	`a2b29db`	fix(xrpl-ledger): F210 — a forward pool hop re-anchors to the reverse pass's cached out
14:57 UTC	xrpl-validator	`b5d35c7`	fix(xrpl-ledger): F211 — a dry rival strand hands the round to the direct pool
14:57 UTC	xrpl-validator	`72af27a`	fix(xrpl-ledger): F212 — a bridged crossing's round is bounded by the deferred line
14:57 UTC	xrpl-validator	`a6bd0eb`	fix(xrpl-ledger): F213 — a bridged crossing's remainder after a pool slice is sixteen digits
14:57 UTC	xrpl-validator	`fd432ed`	fix(xrpl-ledger): F214 — the bridged round's deferred bound is per iteration
14:57 UTC	xrpl-validator	`70d0675`	fix(xrpl-ledger): F215 — a bridged crossing's strands die for the rest of the crossing
14:57 UTC	xrpl-validator	`bb922f1`	fix(xrpl-ledger): F216 — a sell-mode offer crossing never re-anchors to its TakerPays
14:57 UTC	xrpl-validator	`1e805e5`	fix(xrpl-ledger): F217 — the payment pot counts down from SendMax; hop zero is bounded by the holding
15:42 UTC	xrpl-validator	`7163cdc`	fix(xrpl-ledger): F218 — a spend that exactly meets the reverse pass runs no forward pass
15:42 UTC	xrpl-validator	`bafa4fd`	test(xrpl-node): fetch_tx_bundle rebuilds later-touched targets from the tx's own meta
16:44 UTC	xrpl-validator	`9795101`	test(xrpl-node): fetch_tx_bundle rebuilds from FinalFields alone
16:44 UTC	xrpl-validator	`65a988a`	fix(xrpl-ledger): F219 — hop zero's gross cap is bounded by the holding
16:44 UTC	xrpl-validator	`3b2f993`	fix(xrpl-ledger): F220 — an input-driven strand's last book hop consumes its carry

Merged	Repo	Commit	Subject
18:31 UTC	xrpl-validator	`26c3b63`	chore(xrpl-ledger): drop the dead initializer clippy flagged on pool_bq_blocked
18:31 UTC	xrpl-validator	`798c6c8`	fix(loop): runner diff mode rebuilds differential_probe unconditionally
18:31 UTC	xrpl-validator	`248bb47`	fix(loop): differential_probe hydrates the pool directory/lines and the taker's pays-side line
18:31 UTC	xrpl-validator	`bd09003`	fix(scripts): fetch_tx_bundle pulls the escrow, channel and check a refused tx names
18:31 UTC	xrpl-validator	`e1e82e7`	fix(xrpl-ledger): F222 — AccountDelete refuses NFT holders and issuers before the directory walk
18:31 UTC	xrpl-validator	`50e32c7`	fix(xrpl-ledger): F221 — a deposit refuses tecFROZEN when the issuer froze the pool's own line
22:17 UTC	xrpl-validator	`3f67177`	fix(xrpl-ledger): F223 — a strand-trimmed ask keeps the limit-quality forgiveness in the pool judge
23:02 UTC	xrpl-validator	`9771135`	fix(xrpl-ledger): F224 — the bridged strand's limit solve carries the in-side transfer fee
23:56 UTC	xrpl-validator	`d4965d4`	fix(xrpl-ledger): F225 — AMMWithdraw refuses tecAMM_BALANCE by rippled's pool-balance guards
02:48 UTC	xrpl-validator	`5d875fe`	chore(xrpl-ledger): markdown bullets in the F225 guard doc (clippy doc_lazy_continuation)
02:48 UTC	xrpl-validator	`8324b31`	fix(xrpl-ledger): F226 — the exhausting fill's remainder folds the saved inputs as a sorted multiset

Findings fixed

- **fix(xrpl-ledger): F198 — the direct walk's tail pool turn iterates like rippled's flow()** (`c1b1e51`, xrpl-validator) rippled's flow() keeps iterating a single-path pool: each pass is one AMMLiquidity::getOffer — maxOffer, 99% of the pool's out at the CURRENT balances — and the next pass starts from the moved pool, until the taker's in is spent, the want is met, or a pass misses limitQuality (`limitOut` sizes it to the limit, the realised quality is judged, "Path rejected by limitQuality" → "All strands dry"). Our tail turn ran …
- **fix(xrpl-ledger): F199 — TrustSet refuses tfSetNoRipple on a negative balance** (`79395eb`, xrpl-validator) TrustSet::doApply: "Cannot set noRipple on a negative balance" — if (bSetNoRipple && !bClearNoRipple) { if ((bHigh ? saHighBalance : saLowBalance) >= beast::zero) uFlagsOut |= (bHigh ? lsfHighNoRipple : lsfLowNoRipple); else return tecNO_PERMISSION; } the first refusal on an existing line, ahead of the reserve check. The sender's balance is the stored (low-perspective) one negated for the high side. do_apply set the …
- **fix(xrpl-ledger): F200 — an in-limited pool slice delivers no more than the remaining want** (`a8439a9`, xrpl-validator) `consume`'s plain in-limit (take_in > rem_gets → re-swap the whole remaining in through swapAssetIn) can land the out ABOVE the remaining want. rippled's forward pass never delivers that surplus: the strand's last DirectStep keeps its REVERSE cache whenever the forward input exceeds it (`setCacheLimiting`, finding 170), so the beneficiary is credited the reverse out — the want — while the pool parts with the whole …
- **fix(xrpl-ledger): F201 — consecutive self-offers are swept in the pass that is still walking** (`7c49aa8`, xrpl-validator) BookStep::forEachOffer runs `tryAMM` ONCE per pass, at the FIRST tip's quality — `if (tryAMM(offers.tip().quality())) do { execOffer(tip) } while (offers.step())` — and `execOffer` removes the taker's own offer before it looks at any amount (limitSelfCrossQuality). So once a pass has removed the self-offer at one level, it steps straight onto the next level's tip: a self-offer there is removed too, and no pool offer …

- **fix(xrpl-ledger): F202 — the bridged walk's remaining want is the entry want less the ascending fold of its saved outs** (`5182de9`, xrpl-validator) rippled's StrandFlow keeps the per-iteration outs in a flat_multiset and re-derives `remainingOut = outReq − sum(savedOuts)` after every iteration (StrandFlow.h:743) — the saved outs folded ASCENDING, each `+=` an STAmount add canonicalised to sixteen digits. The direct walk has carried that since finding 173; the bridged walk still subtracted each fill's out from the running remainder.
- **fix(xrpl-ledger): F203 — a payment strand's quality function takes a pool only where its fee-inclusive spot beats the tip** (`955de48`, xrpl-validator) rippled's `limitOut` (StrandFlow.h) composes each step's quality function, and a BookStep's is the pool's (linear) only when `tipOfferQualityF` gets an AMM offer that beats the tip: `getAMMOffer(view, qualityThreshold(lob))`, where a PAYMENT step's `qualityThreshold` is the tip itself (`BookPaymentStep::qualityThreshold { return lobQuality; }`, BookStep.cpp:320). The `qualityThreshold_ > lobQuality → nullopt` …
- **fix(xrpl-ledger): F204 — an input-limited payment strand drives its middle hops by the carry** (`888ecec`, xrpl-validator) Once the reverse pass asks more input than the strand has, rippled's forward pass is driven by the INPUT at every book step — `BookStep::fwdImp` hands `limitStepIn` the whole carry, and the reverse amounts are reinstated only on an exact cache hit — and only the closing DirectStep clamps the delivery to the reverse want (finding 170). Each hop here bought exactly the reverse want and left the carry's rounding …
- **fix(xrpl-ledger): F205 — a bridge admitted by its multipath bound runs only if its single-path bound clears the limit too** (`2b27ffd`, xrpl-validator) flow()'s activateNext prices a strand's qualityUpperBound under the CURRENT multiPath flag. A round that admits the bridge as one of two strands prices it under multiPath — leg A by the pool's fib slice at its spot, leg B by its tip — but once that verdict leaves the bridge alone, the next activateNext re-prices it under multiPath=false: leg A by the pool offer ANCHORED on the USD/XRP tip, its in-side transfer rate …
- **fix(xrpl-ledger): F206 — a pool slice's quality is getRate-rounded, as rippled's Quality{amounts}** (`7aaf827`, xrpl-validator) An AMM fib offer's quality is rippled's `Quality{amounts}` = `getRate(out, in)` = `divide(in, out)`: floor(in·10^17/out) + 5, then canonicalised to sixteen digits under Number's default ToNearest. `slice_rate` divided to nearest instead — one ulp apart on the rate's last digit — and a leg fill limited by the other leg (`ceilInStrict`: the input divided by that rate, floored) inherits the ulp.
- **fix(xrpl-ledger): F208 — a crossing's pool turn is bounded by the taker's live line** (`bdc467e`, xrpl-validator) rippled's DirectStepI reverse pass bounds the taker's in by `PaymentSandbox::balanceHookIOU` = min(live line, original − Σdebits). The direct walk's `rem_gets` is the second figure — the fold of the spends — and the line, adjusted sequentially at sixteen digits, can sit an ulp or two under it. The CLOB fill already takes the line (finding 174); the pool turn sized its slice by `rem_gets` alone.
- **fix(xrpl-ledger): F209 — an IOU difference at Number's cusp rounds the coarse mantissa** (`f0642ec`, xrpl-validator) rippled 3.3.0 subtracts IOUs through `Number` (STAmount.cpp `operator+` → `IOUAmount::operator+=` → `Number::operator+=`) at the sixteen-digit "Small" mantissa scale — Rules.cpp `setCurrentTransactionRules` selects the nineteen-digit scales only under SingleAssetVault or LendingProtocol, neither live on mainnet — where the cusp-rounding fix is Disabled. The operand with the smaller exponent is truncated to the …
- **fix(xrpl-ledger): F210 — a forward pool hop re-anchors to the reverse pass's cached out** (`a2b29db`, xrpl-validator) rippled's `BookStep::fwdImp`, when "the step produced more output in the forward pass than the reverse pass while consuming the same input (or less)", re-prices the offer for the cached out — `limitStepOut(cache_-> out)`, for a single-path pool `AMMOffer::limitOut` = `swapAssetOut` — and when

the input that requires equals the input provided it consumes that input and produces exactly the output the reverse pass …

- **fix(xrpl-ledger): F211 — a dry rival strand hands the round to the direct pool** (`b5d35c7`, xrpl-validator) An AMM offer is gated by ITS OWN book's CLOB tip alone: `AMMLiquidity::getOffer(sb, clobQuality)` returns nullopt when `Quality{amounts} < clobQuality`, and `clobQuality` is the tip of the book the step walks (`tryAMM(lobQuality)`). Which strand runs is decided elsewhere: `flow()` tries the active strands best-bound first, and the direct strand runs whenever the bridge ahead of it is dry, rejected or spent. The …
- **fix(xrpl-ledger): F212 — a bridged crossing's round is bounded by the deferred line** (`72af27a`, xrpl-validator) The direct pool strand's DirectStep is bounded like every other: `PaymentSandbox::balanceHookIOU` = min(live line, original − Σdebits) (finding 197 for the exhausting book fills; finding 208 for the direct walk's pool turns). The bridged crossing's pool turn — the direct pool competing with the bridge each round — handed the pool the walk's own remainder instead, so the slice that exhausts the crossing's funds sold …
- **fix(xrpl-ledger): F213 — a bridged crossing's remainder after a pool slice is sixteen digits** (`a6bd0eb`, xrpl-validator) A crossing's remaining in is an STAmount: rippled's `remainingIn` is an IOUAmount and the DirectStep's bound a line balance, both sixteen digits (finding 118 keeps the book fills' remainder that way). The bridged crossing's pool turn kept the exact difference instead.
- **fix(xrpl-ledger): F214 — the bridged round's deferred bound is per iteration** (`fd432ed`, xrpl-validator) The deferred bound (finding 212) is per iteration: rippled's strand keeps its own `remainingIn` and the DirectStep re-reads the line and the deferred table each pass. Persisting the bound into the walk's remaining in left the chain a unit short of the live line from the first bounded round on, and at the exhausting round the chain undercut the deferred bound itself. The chain now continues from what the round …
- **fix(xrpl-ledger): F215 — a bridged crossing's strands die for the rest of the crossing** (`70d0675`, xrpl-validator) rippled's `ActiveStrands` (StrandFlow.h) rebuilds the next iteration's strand list only from the strands that produced this iteration (`push`) and the untried remainder behind the winner (`pushRemainingCurToNext`); `activateNext` drops a pending strand whose upper bound misses limitQuality when more than one is pending, and never re-checks a lone one. A strand dropped or run dry is gone for the rest of the crossing, …
- **fix(xrpl-ledger): F216 — a sell-mode offer crossing never re-anchors to its TakerPays** (`bb922f1`, xrpl-validator) Finding 210's forward re-anchor — deliver the reverse pass's cached out when the input that requires equals the input provided — has no cached out to re-anchor to in a SELL crossing. `CreateOffer::flowCross` replaces a sell's deliver amount with the largest representable one ("we are selling, so we will accept *more* than the offer specified", OfferCreate.cpp:416-441), so the reverse pass's `cache_->out` is …
- **fix(xrpl-ledger): F217 — the payment pot counts down from SendMax; hop zero is bounded by the holding** (`1e805e5`, xrpl-validator) rippled's `remainingIn` counts down from SendMax — `sendMax − sum(savedIns)` (StrandFlow.h) — and the sender's holding enters only through the DirectStep's per-iteration bound, `balanceHookIOU` = min(live line, original − Σdebits). The payment engine clamped its pot to the holding up front, so the fold's remainder, not the line, sized the exhausting fill.
- **fix(xrpl-ledger): F218 — a spend that exactly meets the reverse pass runs no forward pass** (`7163cdc`, xrpl-validator) `flow()` (StrandFlow.h) walks a strand forward only from its limiting step. When the reverse pass finds nothing limiting — every step meets its request and the strand's in does not exceed SendMax — there is no forward pass at all: the reverse-sized amounts, already applied in the reverse pass's sandbox, are the strand's result. Our payment engine treated hop 0 spending its whole input as the mark of an input-limited …

- **fix(xrpl-ledger): F219 — hop zero's gross cap is bounded by the holding** (`65a988a`, xrpl-validator) Finding 217 let the payment pot count down from SendMax and bounded hop zero by the sender's holding — but the holding is GROSS and the hop's budget is NET of the spend leg's transfer rate, so on a rated leg the bound never bit, and the verbatim gross cap the exhausting fill debits (finding 118) was SendMax itself.
- **fix(xrpl-ledger): F220 — an input-driven strand's last book hop consumes its carry** (`3b2f993`, xrpl-validator) An input-limited strand's forward pass drives every book step by input, the last one included: `fwdImp` hands `limitStepIn` the whole carry, and the closing DirectStep clamps the delivery to the reverse want (`setCacheLimiting`, finding 170), the excess redeemed against the issuer. Finding 204 walked the middle hops that way but kept the last hop out-limited to its want, so a last CLOB fill stopped short of the …
- **fix(xrpl-ledger): F222 — AccountDelete refuses NFT holders and issuers before the directory walk** (`e1e82e7`, xrpl-validator) AccountDelete.cpp:256-303 decides four things before it reads the owner directory, and we modelled only the Sequence rule — after the walk: MintedNFTokens != BurnedNFTokens → tecHAS_OBLIGATIONS any NFTokenPage in [nftokenPageMin, nftokenPageMax] → tecHAS_OBLIGATIONS Sequence + 255 > view.seq() → tecTOO_SOON FirstNFTokenSequence + MintedNFTokens + 255 > seq → tecTOO_SOON
- **fix(xrpl-ledger): F221 — a deposit refuses tecFROZEN when the issuer froze the pool's own line** (`50e32c7`, xrpl-validator) AMMDeposit.cpp:330-346 `checkAmount`, while fixCleanup3_3_0 is not live: after the strong requireAuth on each deposited amount, rippled runs checkFrozen(view, ammAccountID, asset) — the AMM account's own trust line frozen by the issuer (or a global freeze) refuses the deposit tecFROZEN even though the depositor's line is clean; withdrawals stay allowed. Our preclaim ran the depositor's requireAuth/checkFrozen for …
- **fix(xrpl-ledger): F223 — a strand-trimmed ask keeps the limit-quality forgiveness in the pool judge** (`3f67177`, xrpl-validator) StrandFlow.h:664-672 trims remainingOut to the quality function's solve and remembers that it did (`adjustedRemOut`); the post-check at :739-741 then forgives a fill up to 1e-7 over limitQuality when — and only when — the request was trimmed. Our payment driver does the trim (`adjusted_ask`) and its own strand judge honours it, but the AMM pool judge in `consume` recomputed the solve, found the incoming request …
- **fix(xrpl-ledger): F224 — the bridged strand's limit solve carries the in-side transfer fee** (`9771135`, xrpl-validator) BookStep::getQualityFunc folds the in-side transfer fee into the first leg's quality function: adjustQualityWithFees(kQOne, prevStepDir) with the taker's DirectStep redeeming makes trIn the gets issuer's TransferRate, and that quality is combined as a CLOB-like constant BEFORE the pool curve (a CLOB tip gets the fee composed into its quality — the same product). Our bridged crossing composed the two pool curves …
- **fix(xrpl-ledger): F225 — AMMWithdraw refuses tecAMM_BALANCE by rippled's pool-balance guards** (`d4965d4`, xrpl-validator) AMMWithdraw.cpp:568-593 `withdraw()` judges the ACTUAL amounts of every mode before anything moves and refuses tecAMM_BALANCE when one side of the pool would be drained but not the other, when every LP token is burned without draining both sides, or when more than the pool holds of either asset would leave. A one-asset withdrawal carries no second amount, and rippled's std::optional compares unequal to any balance, …
- **fix(xrpl-ledger): F226 — the exhausting fill's remainder folds the saved inputs as a sorted multiset** (`8324b31`, xrpl-validator) flow() recomputes `remainingIn = sendMax - sum(savedIns)` after every iteration (StrandFlow.h:745), and `savedIns` is a sorted multiset: the sixteen-digit fold runs in ASCENDING order — the same shape finding 202 gave `savedOuts` for remainingOut. Our crossing walks rebuilt the exhausting fill's remainder from `in_gross_spent`, an insertion-order Σ, and the two folds round apart once in a while.

Other engineering changes

- **fix(loop): runner diff mode rebuilds differential_probe unconditionally** (`798c6c8`, xrpl-validator) cmd_diff built the native-leg binary only when it was absent, so loop/harness/target/debug/differential_probe sat at 2026-09-05 23:06 while the harness tree tracked main — the hourly scout judged every "diff" probe with a two-day-old engine. On 2026-09-07 that produced four phantom AMMWithdraw tecAMM_INVALID_TOKENS finds (#106816232 D45A17916FD1, #106821332 A56150E02425, #106822082 B857D3D4FF66, #106826178 …
- **fix(loop): differential_probe hydrates the pool directory/lines and the taker's pays-side line** (`248bb47`, xrpl-validator) Two hydration gaps in the scout's native leg, each a phantom TER refusal on state the live shadow and probe_bundle judge correctly: - AMMDeposit/AMMWithdraw: load_amm_prestate fetched only the pool's directory root and tail page and none of the other LPs' lines, so the engine's isOnlyLiquidityProvider walk (verify_and_adjust_lp_token_balance skips unhydrated entries) saw a sole LP, snapped the line against …
- **fix(scripts): fetch_tx_bundle pulls the escrow, channel and check a refused tx names** (`bd09003`, xrpl-validator) A tec EscrowFinish/EscrowCancel (or a refused PaymentChannel/*Check* claim) touches nothing but the fee, so the object it names never reaches the meta and the bundle lacked it; the probe then answered tecNO_TARGET for whatever mainnet actually decided. #106743443 44A81646DF43: an EscrowFinish two seconds before FinishAfter — mainnet tecNO_PERMISSION, ours tecNO_TARGET with the escrow unhydrated. keylet::escrow = …

Tests and campaigns

- **test(xrpl-node): close the F201 vector test — the F201/F202 merge resolution dropped its brace** (`0c4d063`, xrpl-validator)
- **test(xrpl-node): fetch_tx_bundle rebuilds later-touched targets from the tx's own meta** (`bafa4fd`, xrpl-validator)
- **test(xrpl-node): fetch_tx_bundle rebuilds from FinalFields alone** (`9795101`, xrpl-validator)

Docs and operations

- **chore(xrpl-ledger): drop the dead initializer clippy flagged on pool_bq_blocked** (`26c3b63`, xrpl-validator)
- **chore(xrpl-ledger): markdown bullets in the F225 guard doc (clippy doc_lazy_continuation)** (`5d875fe`, xrpl-validator)

Independence program

No independence record for UTC day 2026-09-07; records begin 2026-10-20.

Network

No ledger records for this day: the reference node holds ledgers from 2026-10-04 22:52 UTC (#107,436,647).

Amendment watch

Carried over unchanged from the original report, which read it from the XRP Ledger when it was generated; the reference node no longer holds this day's ledgers.

Read from the XRP Ledger: the Amendments object as of the last ledger of the day, and the day's EnableAmendment pseudo-transactions.

Amendment	Majority since	Activation due
fixCleanup3_3_0	2026-08-28 11:15 UTC	2026-09-11 11:15 UTC

Amendments enabled on mainnet: 93.

Open items

- Receipt R-01 (#106,815,089): open, under investigation.
- Receipt R-02 (#106,815,131): open, under investigation.
- Receipt R-04 (#106,815,284): open, under investigation.
- Receipt R-12 (#106,818,252): open, under investigation.
- Receipt R-14 (#106,819,769): open, under investigation.
- Receipt R-19 (#106,823,313): open, under investigation.
- Receipt R-20 (#106,823,726): open, under investigation.
- Receipt R-25 (#106,823,826): open, under investigation.
- Receipt R-26 (#106,823,836): open, under investigation.
- Receipt R-27 (#106,823,837): open, under investigation.
- Receipt R-28 (#106,823,869): open, under investigation.
- Receipt R-29 (#106,823,870): open, under investigation.
- Receipt R-33 (#106,823,900): open, under investigation.
- Receipt R-34 (#106,823,926): open, under investigation.
- Receipt R-35 (#106,823,927): open, under investigation.
- Receipt R-36 (#106,823,941): open, under investigation.
- Receipt R-37 (#106,823,945): open, under investigation.
- Receipt R-40 (#106,824,971): open, under investigation.
- Receipt R-41 (#106,825,264): open, under investigation.
- Receipt R-45 (#106,826,383): open, under investigation.
- Receipt R-47 (#106,826,878): open, under investigation.
- Receipt R-48 (#106,826,903): open, under investigation.
- Receipt R-49 (#106,826,942): open, under investigation.
- Receipt R-50 (#106,826,963): open, under investigation.
- Receipt R-51 (#106,826,964): open, under investigation.
- Receipt R-52 (#106,827,527): open, under investigation.
- Receipt R-53 (#106,827,576): open, under investigation.
- Receipt R-54 (#106,827,595): open, under investigation.

- Receipt R-55 (#106,827,601): open, under investigation.

Corrections to earlier reports

None.

XRPL research

Automated research — candidate amendment ideas

Ideas generated and prior-art-checked by the XRPL Amendment Board, an automated research tool (read-only board: iceland.halcyon-names.io). They are exploration leads, not proposals.

- **Scan-Key Stealth Payment Destinations** (privacy) Add an optional sfStealthScanKey (33-byte compressed pubkey) to AccountRoot, settable via a new AccountSet flag asfStealthAddress, and add a Payment f
- **Concentrated AMM Ranges with Fee Tiers** (amm-defi) Add a new owner-reserved ledger object ltAMM_POSITION (keylet::ammPosition(ammID, owner, seq)) holding sfTickLower, sfTickUpper (int32 price ticks on
- **Header-Proof Cross-Chain Message Bridge** (interop) Extend the Bridge object with an ltXCHAIN_HEADER_STORE tracking sfLastVerifiedLedgerHash/Seq and sfCounterpartyUNLHash, populated by a new XChainHeade

Appendix — commits

Committed	Merged into main	Repo	Commit	Subject
09-07 04:30	09-07 04:30	xrpl-validator	`c1b1e51`	fix(xrpl-ledger): F198 — the direct walk's tail pool turn iterates like rippled's flow()
09-07 05:16	09-07 05:16	xrpl-validator	`79395eb`	fix(xrpl-ledger): F199 — TrustSet refuses tfSetNoRipple on a negative balance
09-07 05:57	09-07 05:57	xrpl-validator	`a8439a9`	fix(xrpl-ledger): F200 — an in-limited pool slice delivers no more than the remaining want
09-07 07:49	09-07 07:49	xrpl-validator	`7c49aa8`	fix(xrpl-ledger): F201 — consecutive self-offers are swept in the pass that is still walking
09-07 08:34	09-07 08:34	xrpl-validator	`5182de9`	fix(xrpl-ledger): F202 — the bridged walk's remaining want is the entry want less the ascending fold of its saved outs
09-07 08:43	09-07 08:43	xrpl-validator	`0c4d063`	test(xrpl-node): close the F201 vector test — the F201/F202 merge resolution dropped its brace
09-07 08:43	09-07 09:37	xrpl-validator	`955de48`	fix(xrpl-ledger): F203 — a payment strand's quality function takes a pool only where its fee-inclusive spot beats the tip
09-07 08:43	09-07 10:20	xrpl-validator	`888ecec`	fix(xrpl-ledger): F204 — an input-limited payment strand drives its middle hops by the carry
09-07 08:43	09-07 11:03	xrpl-validator	`2b27ffd`	fix(xrpl-ledger): F205 — a bridge admitted by its multipath bound runs only if its single-path bound clears the limit too
09-07 08:43	09-07 11:45	xrpl-validator	`7aaf827`	fix(xrpl-ledger): F206 — a pool slice's quality is getRate-rounded, as rippled's Quality{amounts}

Committed	Merged into main	Repo	Commit	Subject
09-07 12:48	09-07 12:49	xrpl-validator	`bdc467e`	fix(xrpl-ledger): F208 — a crossing's pool turn is bounded by the taker's live line
09-07 13:14	09-07 13:35	xrpl-validator	`f0642ec`	fix(xrpl-ledger): F209 — an IOU difference at Number's cusp rounds the coarse mantissa
09-07 13:16	09-07 13:35	xrpl-validator	`a2b29db`	fix(xrpl-ledger): F210 — a forward pool hop re-anchors to the reverse pass's cached out
09-07 13:56	09-07 14:57	xrpl-validator	`b5d35c7`	fix(xrpl-ledger): F211 — a dry rival strand hands the round to the direct pool
09-07 14:05	09-07 14:57	xrpl-validator	`72af27a`	fix(xrpl-ledger): F212 — a bridged crossing's round is bounded by the deferred line
09-07 14:08	09-07 14:57	xrpl-validator	`a6bd0eb`	fix(xrpl-ledger): F213 — a bridged crossing's remainder after a pool slice is sixteen digits
09-07 14:17	09-07 14:57	xrpl-validator	`fd432ed`	fix(xrpl-ledger): F214 — the bridged round's deferred bound is per iteration
09-07 14:31	09-07 14:57	xrpl-validator	`70d0675`	fix(xrpl-ledger): F215 — a bridged crossing's strands die for the rest of the crossing
09-07 14:49	09-07 14:57	xrpl-validator	`bb922f1`	fix(xrpl-ledger): F216 — a sell-mode offer crossing never re-anchors to its TakerPays
09-07 14:56	09-07 14:57	xrpl-validator	`1e805e5`	fix(xrpl-ledger): F217 — the payment pot counts down from SendMax; hop zero is bounded by the holding
09-07 15:08	09-07 15:42	xrpl-validator	`7163cdc`	fix(xrpl-ledger): F218 — a spend that exactly meets the reverse pass runs no forward pass
09-07 15:14	09-07 15:42	xrpl-validator	`bafa4fd`	test(xrpl-node): fetch_tx_bundle rebuilds later-touched targets from the tx's own meta
09-07 16:38	09-07 16:44	xrpl-validator	`9795101`	test(xrpl-node): fetch_tx_bundle rebuilds from FinalFields alone
09-07 16:41	09-07 16:44	xrpl-validator	`65a988a`	fix(xrpl-ledger): F219 — hop zero's gross cap is bounded by the holding
09-07 16:44	09-07 16:44	xrpl-validator	`3b2f993`	fix(xrpl-ledger): F220 — an input-driven strand's last book hop consumes its carry
09-07 16:47	09-07 18:31	xrpl-validator	`26c3b63`	chore(xrpl-ledger): drop the dead initializer clippy flagged on pool_bq_blocked
09-07 18:04	09-07 18:31	xrpl-validator	`798c6c8`	fix(loop): runner diff mode rebuilds differential_probe unconditionally
09-07 18:16	09-07 18:31	xrpl-validator	`248bb47`	fix(loop): differential_probe hydrates the pool directory/lines and the taker's pays-side line
09-07 18:30	09-07 18:31	xrpl-validator	`bd09003`	fix(scripts): fetch_tx_bundle pulls the escrow, channel and check a refused tx names
09-07 18:30	09-07 18:31	xrpl-validator	`e1e82e7`	fix(xrpl-ledger): F222 — AccountDelete refuses NFT holders and issuers before the directory walk
09-07 18:30	09-07 18:31	xrpl-validator	`50e32c7`	fix(xrpl-ledger): F221 — a deposit refuses tecFROZEN when the issuer froze the pool's own line
09-07 22:16	09-07 22:17	xrpl-validator	`3f67177`	fix(xrpl-ledger): F223 — a strand-trimmed ask keeps the limit-quality forgiveness in the pool judge
09-07 22:37	09-07 23:02	xrpl-validator	`9771135`	fix(xrpl-ledger): F224 — the bridged strand's limit solve carries the in-side transfer fee
09-07 23:56	09-07 23:56	xrpl-validator	`d4965d4`	fix(xrpl-ledger): F225 — AMMWithdraw refuses tecAMM_BALANCE by rippled's pool-balance guards
09-08 00:07	09-08 02:48	xrpl-validator	`5d875fe`	chore(xrpl-ledger): markdown bullets in the F225 guard doc (clippy doc_lazy_continuation)

Committed	Merged into main	Repo	Commit	Subject
09-08 02:47	09-08 02:48	xrpl-validator	`8324b31`	fix(xrpl-ledger): F226 — the exhausting fill's remainder folds the saved inputs as a sorted multiset

Times in UTC. A commit appears on the day it was committed to main or merged into it.